Collection Frame Work – method implementation

1.ArrayList

Title : Demonstration of ArrayList Methods in Java

Aim : To write a Java program to demonstrate the important methods of the ArrayList class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware **:**

Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create an ArrayList of integers.
2. Add elements using add().
3. Insert an element using add(index, element).
4. Access elements using get().
5. Replace an element using set().
6. Remove elements using both remove(index) and remove(Object).
7. Check elements using contains().
8. Find the size using size().
9. Check whether the list is empty using isEmpty().
10. Find positions using indexOf() and lastIndexOf().
11. Sort the list using sort().
12. Remove all elements using clear().
13. Display the results.

UML Diagram :

ArrayListDemo

|

|. |. |

Add() get() remove()

|. |. |. |. |

Set() contains() clear() Sorting searching

Source Code :

```
import java.util.ArrayList;
import java.util.Comparator;
public class ArrayListDemo {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        // Adding elements
        numbers.add(30);
        numbers.add(10);
        numbers.add(20);
        numbers.add(10);
        System.out.println(“Original List: “ + numbers);
        // Adding element at a specific index
        numbers.add(1, 40);
        System.out.println(“After add(index, element): “ + numbers);
        // Getting an element
        System.out.println(“Element at index 2: “ + numbers.get(2));
        // Replacing an element
        numbers.set(2, 50);
        System.out.println(“After set(): “ + numbers);
        // Removing using index
        Numbers.remove(1);
        System.out.println(“After remove(index): “ + numbers);
        // Removing using object
```

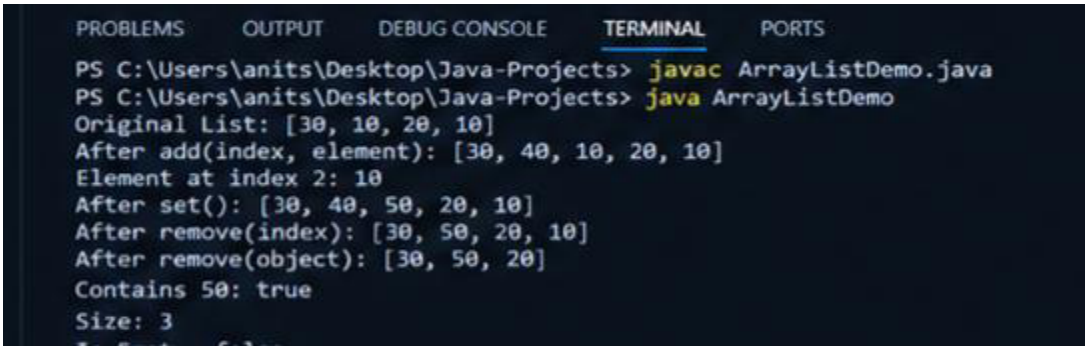
```
        numbers.remove(Integer.valueOf(10));
        System.out.println("After remove(object): " + numbers);
        // Checking an element
        System.out.println("Contains 50: " + numbers.contains(50));
        // Size
        System.out.println("Size: " + numbers.size());
        // Checking empty
        System.out.println("Is Empty: " + numbers.isEmpty());
        // Finding first occurrence
        System.out.println("First index of 10: " + numbers.indexOf(10));
        // Finding last occurrence
        System.out.println("Last index of 10: " + numbers.lastIndexOf(10));
        // Sorting
        numbers.sort(Comparator.naturalOrder());
        System.out.println("Sorted List: " + numbers);
        // Clearing list
        numbers.clear();
        System.out.println("After clear(): " + numbers);
    }
}
```

Compilation & Execution Commands :

Javac ArrayListDemo.java

Java ArrayListDemo

Output screenshot:



Result : Thus, the important methods of the ArrayList class were successfully demonstrated.

2. Linked list :

Title: Demonstration of LinkedList Methods in Java

Aim : To write a Java program to demonstrate the important methods of the LinkedList class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

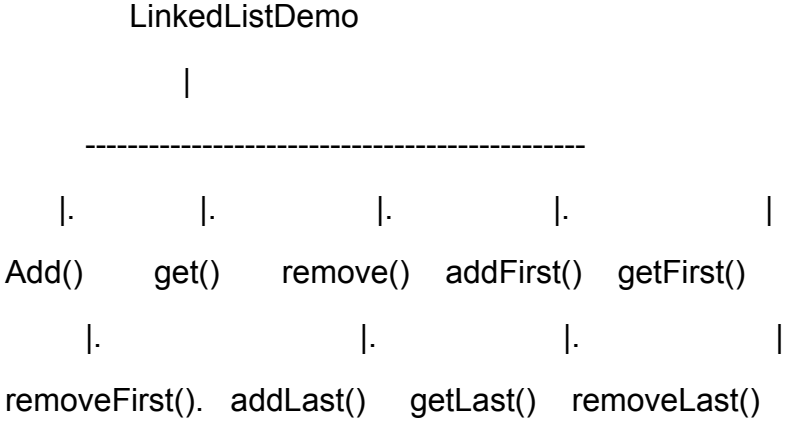
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a LinkedList.
2. Add elements using add().
3. Add elements at the beginning and end using addFirst() and addLast().
4. Access elements using get(), getFirst() and getLast().
5. Remove elements using remove(), removeFirst() and removeLast().
6. Use offer(), poll() and peek().
7. Display the results.

UML Diagram :



|

Offer() → peek() → poll()

Source Code :

```
import java.util.LinkedList;
public class LinkedListDemo {
    public static void main(String[] args) {
       LinkedList<String> names = new LinkedList<>();
        // Adding elements
        names.add(“Anu”);
        names.add(“Kiran”);
        names.add(“Meena”);
        System.out.println(“Original List: “ + names);
        // Adding at beginning and end
        names.addFirst(“Rahul”);
        names.addLast(“Priya”);
        System.out.println(“After addFirst() and addLast(): “ + names);
        // Accessing elements
        System.out.println(“Element at index 2: “ + names.get(2));
        System.out.println(“First Element: “ + names.getFirst());
        System.out.println(“Last Element: “ + names.getLast());
        // Removing elements
        names.remove(2);
        System.out.println(“After remove(index): “ + names);
        names.remove(“Kiran”);
        System.out.println(“After remove(object): “ + names);
        names.removeFirst();
        System.out.println(“After removeFirst(): “ + names);
```

```
        names.removeLast();
        System.out.println("After removeLast(): " + names);
        // Queue operations
        names.offer("Sita");
        names.offer("Ravi");
        System.out.println("After offer(): " + names);
        System.out.println("Peek: " + names.peek());
        System.out.println("Poll: " + names.poll());
        System.out.println("After poll(): " + names);
    }
}
```

Compilation & Execution Commands :

Javac LinkedListDemo.java

Java LinkedListDemo

Output screenshot:



Result : Thus, the important methods of the LinkedList class were successfully demonstrated.

3. Vector :

Title : Demonstration of Vector Methods in Java

Aim : To write a Java program to demonstrate the important methods of the Vector class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

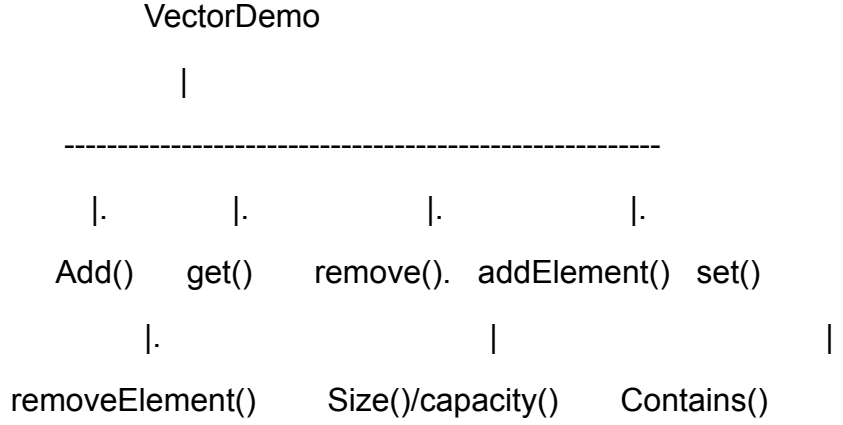
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a Vector.
2. Add elements using add() and addElement().
3. Access an element using get().
4. Replace an element using set().
5. Remove elements using remove() and removeElement().
6. Display size and capacity.
7. Check an element using contains().

UML Diagram



Source Code :

```
import java.util.Vector;
public class VectorDemo {
    public static void main(String[] args) {
        Vector<String> subjects = new Vector<>();
        // Adding elements
        subjects.add(“Java”);
        subjects.add(“C”);
        subjects.addElement(“Python”);
        System.out.println(“Vector: “ + subjects);
        // Accessing element
        System.out.println(“Element at index 1: “ + subjects.get(1));
        // Replacing element
        subjects.set(1, “C++”);
        System.out.println(“After set(): “ + subjects);
        // Removing by index
        subjects.remove(0);
        System.out.println(“After remove(index): “ + subjects);
        // Adding another element
        subjects.add(“JavaScript”);
        // Removing by object
        subjects.removeElement(“Python”);
        System.out.println(“After removeElement(): “ + subjects);
        // Size and capacity
        System.out.println(“Size: “ + subjects.size());
        System.out.println(“Capacity: “ + subjects.capacity());
        // Checking element
        System.out.println(“Contains JavaScript: “
```

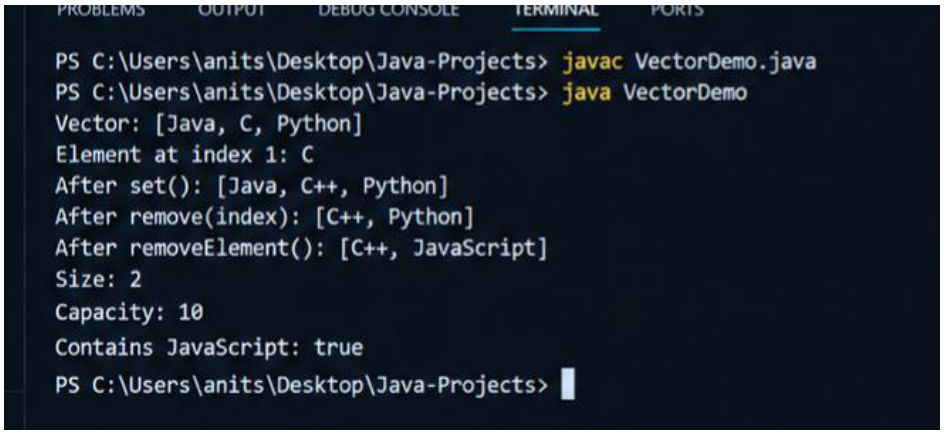
```
            + subjects.contains("JavaScript"));
    }
}
```

Compilation & Execution Commands :

Javac VectorDemo.java

Java VectorDemo

Output screenshot:



Result : Thus, the important methods of the Vector class were successfully demonstrated.

4. Stack

Title: Demonstration of Stack Methods in Java

Aim : To write a Java program to demonstrate the important methods of the Stack class.

Software requirements:

Software :

Java JDK

Eclipse IDE / Command Prompt

Hardware :

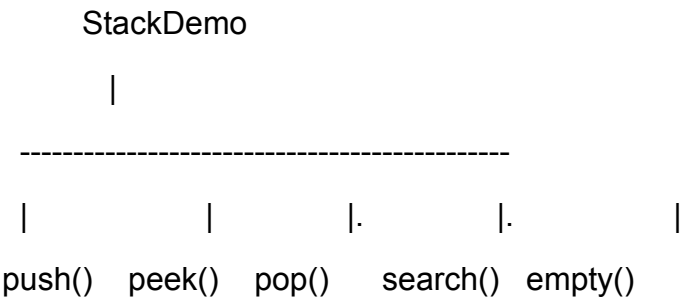
Computer/Laptop

Minimum 4 GB RAM

Algorithm

1. Create a Stack.
2. Add elements using push().
3. Display the top element using peek().
4. Remove the top element using pop().
5. Check whether the stack is empty using empty().
6. Find an element position using search().

UML Diagram :



Source Code :

```
import java.util.Stack;

public class StackDemo {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
        // Adding elements
        stack.push(10);
        stack.push(20);
        stack.push(30);
        System.out.println("Stack: " + stack);
        // Viewing top element
        System.out.println("Top element: " + stack.peek());
        // Removing top element
        System.out.println("Popped element: " + stack.pop());
        System.out.println("Stack after pop: " + stack);
        // Checking empty
```

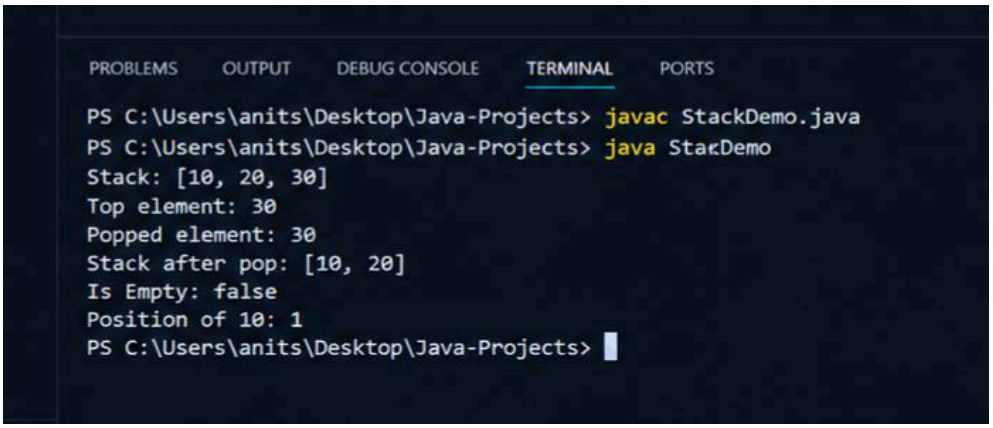
```
        System.out.println("Is Empty: " + stack.empty());
        // Searching element
        System.out.println("Position of 10: " + stack.search(10));
    }
}
```

Compilation & Execution Commands :

javac StackDemo.java

java StackDemo

Output screenshot:



Result :

Thus, the important methods of the Stack class were successfully demonstrated.

5. HashSet

Title : Demonstration of HashSet Methods in Java

Aim : To write a Java program to demonstrate the important methods of the HashSet class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

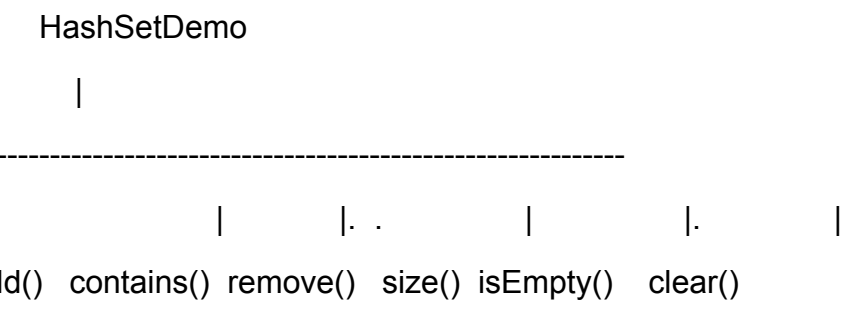
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a HashSet.
2. Add elements using add().
3. Check an element using contains().
4. Find the size using size().
5. Check whether it is empty using isEmpty().
6. Remove an element using remove().
7. Clear the set using clear().

UML Diagram :



Source Code :

```
import java.util.HashSet;

public class HashSetDemo {

    public static void main(String[] args) {

        HashSet<String> cities = new HashSet<>();

        // Adding elements

        cities.add("Delhi");

        cities.add("Mumbai");

        cities.add("Chennai");

        cities.add("Delhi");

       System.out.println("HashSet: " + cities);

        // Checking element

        System.out.println("Contains Mumbai: "
```

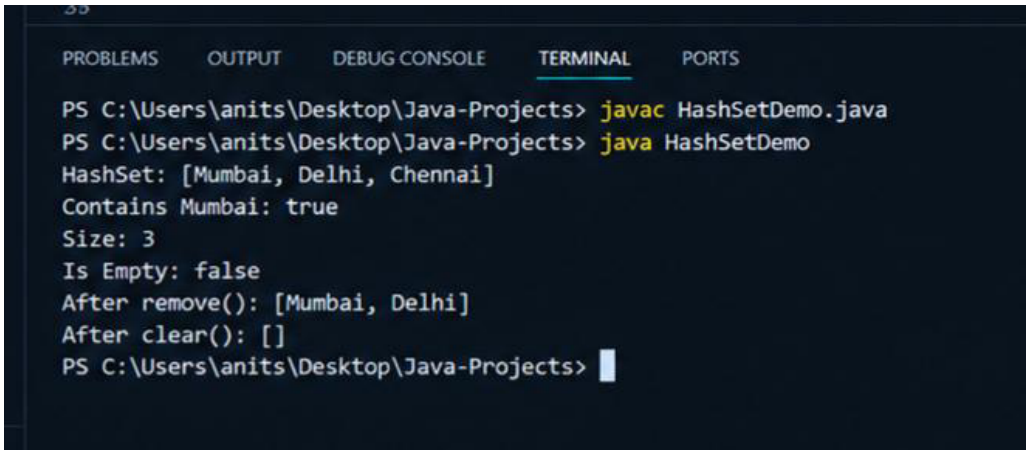
```
            + cities.contains("Mumbai"));
        // Size
        System.out.println("Size: " + cities.size());
        // Checking empty
        System.out.println("Is Empty: " + cities.isEmpty());
        // Removing element
        cities.remove("Chennai");
        System.out.println("After remove(): " + cities);
        // Clearing set
        cities.clear();
        System.out.println("After clear(): " + cities);
    }
}
```

Compilation & Execution Commands :

javac HashSetDemo.java

java HashSetDemo

Output :



Result : Thus, the important methods of the HashSet class were successfully demonstrated.

6. LinkedHashSet :

Title : Demonstration of LinkedHashSet Methods in Java

Aim : To write a Java program to demonstrate the important methods of the LinkedHashSet class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

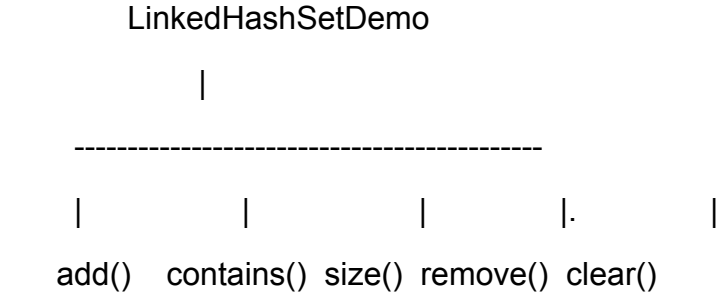
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a LinkedHashSet.
2. Add elements using add().
3. Check an element using contains().
4. Find the number of elements using size().
5. Remove an element using remove().
6. Clear all elements using clear().
7. Display the set after each operation.

UML Diagram :



Source Code :

```
import java.util.LinkedHashSet;

public class LinkedHashSetDemo {
```

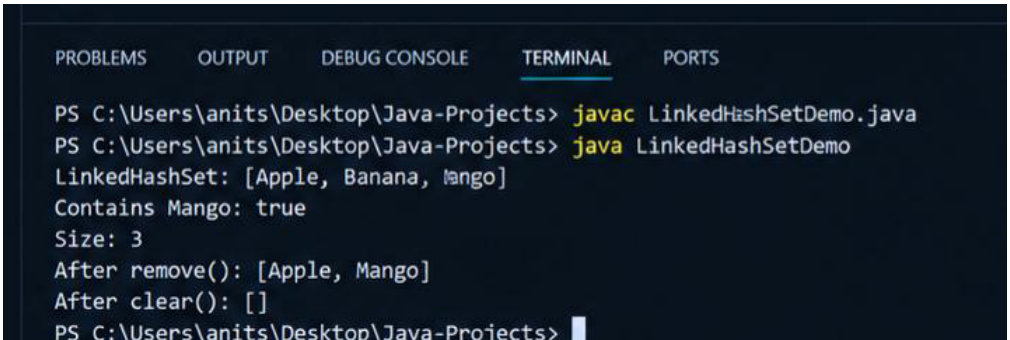
```
    public static void main(String[] args) {
        LinkedHashSet<String> fruits = new LinkedHashSet<>();
       // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");
        fruits.add("Apple");
        System.out.println("LinkedHashSet: " + fruits);
        // Checking an element
        System.out.println("Contains Mango: "
                + fruits.contains("Mango"));
        // Finding size
        System.out.println("Size: " + fruits.size());
        // Removing an element
        fruits.remove("Banana");
        System.out.println("After remove(): " + fruits);
        // Clearing all elements
        fruits.clear();
        System.out.println("After clear(): " + fruits);
    }
}
```

Compilation & Execution Commands :

javac LinkedHashSetDemo.java

java LinkedHashSetDemo

Output screenshot:



Result :

Thus, the important methods of the LinkedHashSet class were successfully demonstrated.

7. TreeSet :

Title : Demonstration of TreeSet Methods in Java

Aim : To write a Java program to demonstrate the important methods of the TreeSet class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

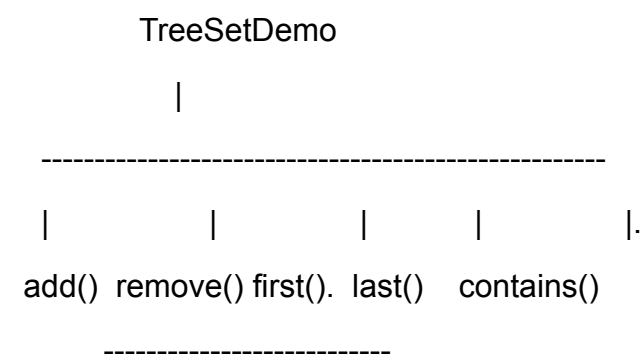
Computer/Laptop

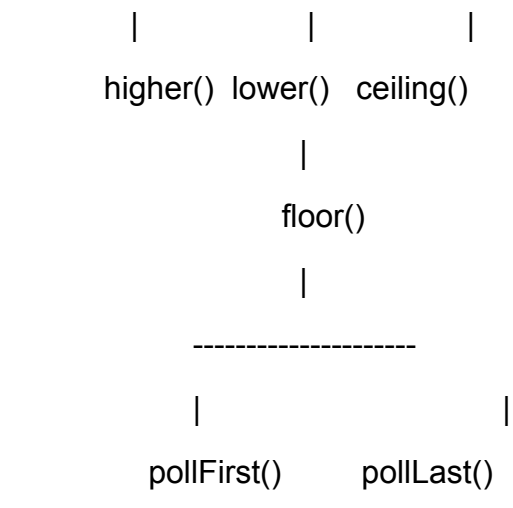
Minimum 4 GB RAM

Algorithm :

1. Create a TreeSet.
2. Add elements using add().
3. Check and remove elements using contains() and remove().
4. Find the first and last elements.
5. Find higher and lower elements.
6. Find ceiling and floor elements.
7. Remove the first and last elements using pollFirst() and pollLast().

UML Diagram :





Source Code :

```
import java.util.TreeSet;
public class TreeSetDemo {
    public static void main(String[] args) {
        TreeSet<Integer> numbers = new TreeSet<>();
        // Adding elements
        numbers.add(30);
        numbers.add(10);
        numbers.add(50);
        numbers.add(20);
        numbers.add(40);
        System.out.println("TreeSet: " + numbers);
        // Checking an element
        System.out.println("Contains 30: " + numbers.contains(30));
        // First and last elements
        System.out.println("First element: " + numbers.first());
        System.out.println("Last element: " + numbers.last());
        // Higher and lower elements
         System.out.println("Higher than 30: " + numbers.higher(30));
```

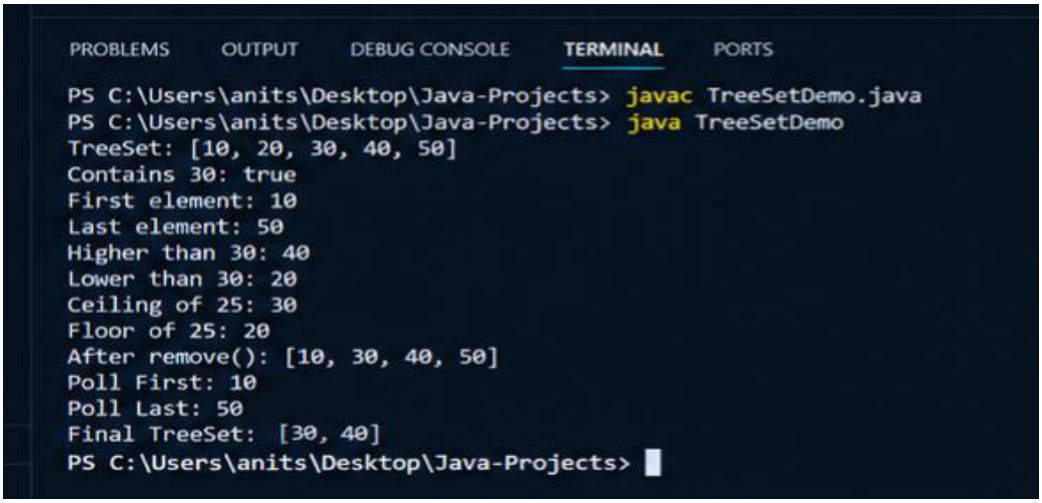
```
        System.out.println("Lower than 30: " + numbers.lower(30));
        // Ceiling and floor elements
        System.out.println("Ceiling of 25: " + numbers.ceiling(25));
        System.out.println("Floor of 25: " + numbers.floor(25));
        // Removing an element
        numbers.remove(20);
        System.out.println("After remove(): " + numbers);
        // Removing first and last elements
        System.out.println("Poll First: " + numbers.pollFirst());
        System.out.println("Poll Last: " + numbers.pollLast());
        System.out.println("Final TreeSet: " + numbers);
    }
}
```

Compilation & Execution Commands :

javac TreeSetDemo.java

java TreeSetDemo

Output screenshot:



Result : Thus, the important methods of the TreeSet class were successfully demonstrated.

8. PriorityQueue

Title : Demonstration of PriorityQueue Methods in Java

Aim : To write a Java program to demonstrate the important methods of the PriorityQueue class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

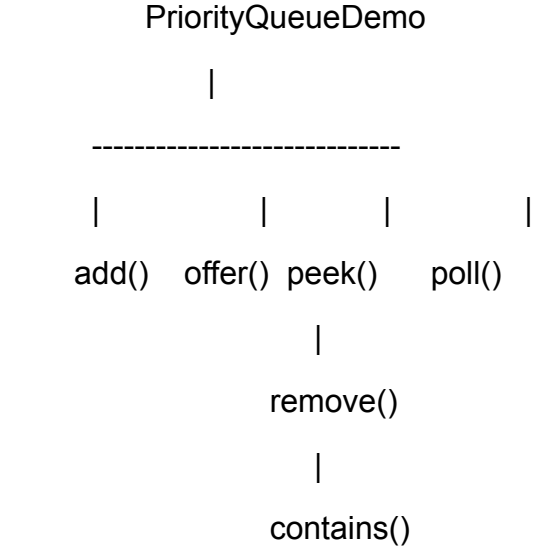
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a PriorityQueue.
2. Insert elements using add() and offer().
3. Display the head using peek().
4. Remove the head using poll().
5. Check an element using contains().
6. Remove a specific element using remove().
7. Display the size using size().

UML Diagram :



|

size()

Source Code :

```
import java.util.PriorityQueue;
public class PriorityQueueDemo {
    public static void main(String[] args) {
        PriorityQueue<Integer> queue = new PriorityQueue<>();
        // Adding elements
        queue.add(40);
        queue.add(20);
        queue.add(50);
        // Adding another element using offer()
        queue.offer(10);
        System.out.println("PriorityQueue: " + queue);
        // Viewing the head
        System.out.println("Peek: " + queue.peek());
        // Checking an element
        System.out.println("Contains 40: " + queue.contains(40));
        // Finding size
        System.out.println("Size: " + queue.size());
        // Removing a specific element
        queue.remove(50);
        System.out.println("After remove(50): " + queue);
        // Removing the highest-priority element
        System.out.println("Poll: " + queue.poll());
        System.out.println("Final Queue: " + queue);
    }
```

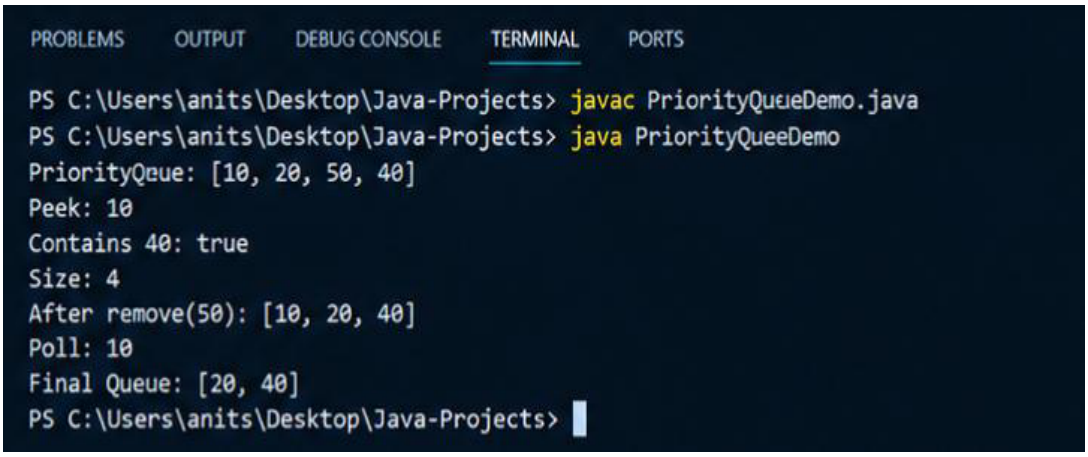
}

Compilation & Execution Commands :

javac PriorityQueueDemo.java

java PriorityQueueDemo

Output screenshot:



Result :

Thus, the important methods of the PriorityQueue class were successfully demonstrated.

9. ArrayDeque

Title : Demonstration of ArrayDeque Methods in Java

Aim : To write a Java program to demonstrate the important methods of the ArrayDeque class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware:

Computer/Laptop

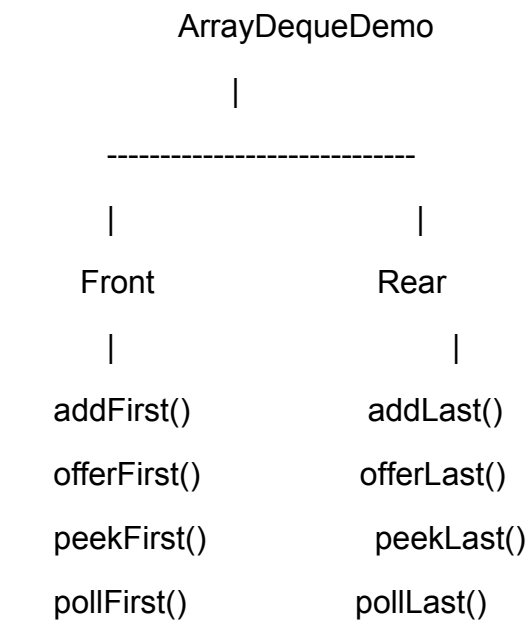
Minimum 4 GB RAM

Algorithm :

1. Create an ArrayDeque.

2. Insert elements using addFirst() and addLast().
3. Insert elements using offerFirst() and offerLast().
4. Display elements using peekFirst() and peekLast().
5. Remove elements using pollFirst() and pollLast().
6. Display the final deque.

UML Diagram :



Source Code :

```
import java.util.ArrayDeque;
public class ArrayDequeDemo {
    public static void main(String[] args) {
        ArrayDeque<String> deque = new ArrayDeque<>();
        // Adding elements at both ends
        deque.addFirst("Java");
      deque.addLast("Python");
        System.out.println("Deque: " + deque);
        // Adding using offer methods
        deque.offerFirst("C");
        deque.offerLast("JavaScript");
```

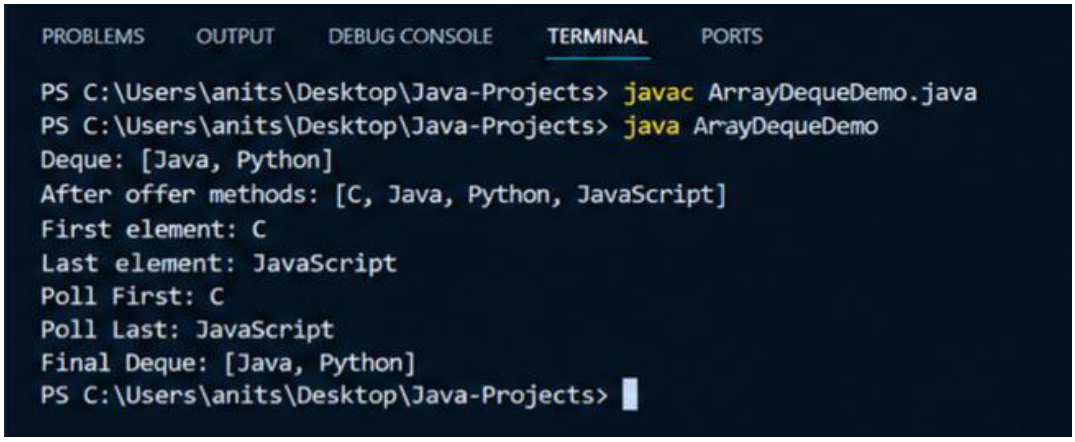
```
        System.out.println("After offer methods: " + deque);
        // Viewing first and last elements
        System.out.println("First element: " + deque.peekFirst());
        System.out.println("Last element: " + deque.peekLast());
        // Removing from both ends
        System.out.println("Poll First: " + deque.pollFirst());
        System.out.println("Poll Last: " + deque.pollLast());
        System.out.println("Final Deque: " + deque);
    }
}
```

Compilation & Execution Commands :

javac ArrayDequeDemo.java

java ArrayDequeDemo

Output screenshot:



Result :

Thus, the important methods of the ArrayDeque class were successfully demonstrated.

10. HashMap

Title : Demonstration of HashMap Methods in Java

Aim : To write a Java program to demonstrate the important methods of the HashMap class.

Software requirements:

Software :

Java JDK

Eclipse IDE / Command Prompt

Hardware :

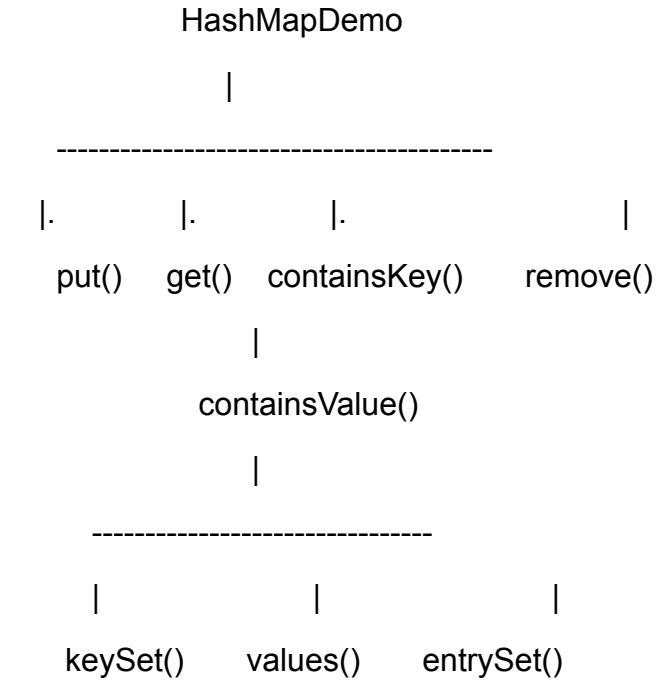
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a HashMap.
2. Add key-value pairs using put().
3. Retrieve values using get().
4. Check keys and values using containsKey() and containsValue().
5. Display keys using keySet().
6. Display values using values().
7. Display mappings using entrySet().
8. Use getOrDefault().
9. Find size and check whether the map is empty.
10. Remove an entry and clear the map.

UML Diagram



|

getOrDefault()

|

size()/isEmpty()

|

clear()

Source Code :

```
import java.util.HashMap;
public class HashMapDemo {
    public static void main(String[] args) {
        HashMap<Integer, String> students = new HashMap<>();
        // Adding key-value pairs
        students.put(101, "Ananya");
        students.put(102, "Kiran");
        students.put(103, "Meera");
        System.out.println("HashMap: " + students);
        // Getting a value
        System.out.println("Value for key 102: " + students.get(102));
        // Checking key and value
        System.out.println("Contains key 101: "
                + students.containsKey(101));
        System.out.println("Contains value Meera: "
                + students.containsValue("Meera"));
        // Displaying keys
        System.out.println("Keys: " + students.keySet());
        // Displaying values
        System.out.println("Values: " + students.values());
```

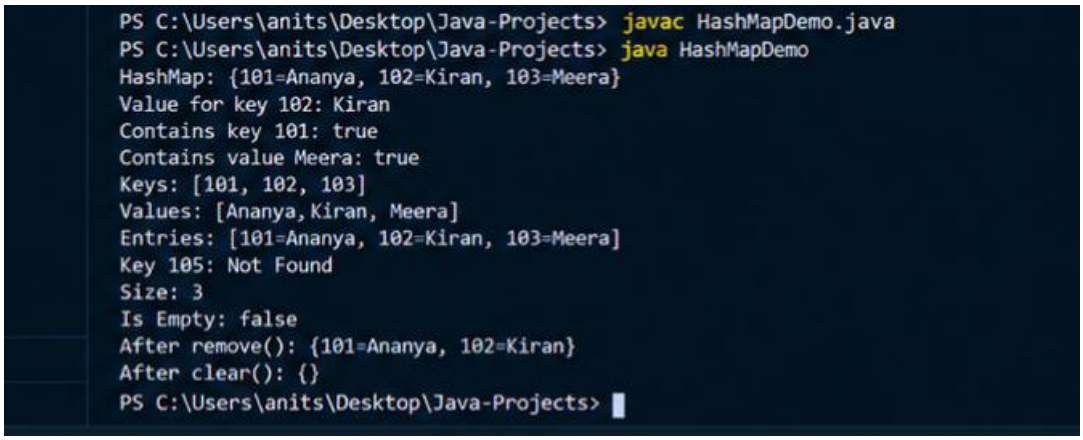
```
        // Displaying entries
        System.out.println("Entries: " + students.entrySet());
       // getOrDefault()
        System.out.println("Key 105: "+ students.getOrDefault(105, "Not Found"));
        // Size and empty check
        System.out.println("Size: " + students.size());
        System.out.println("Is Empty: " + students.isEmpty());
        // Removing an entry
        students.remove(103);
        System.out.println("After remove(): " + students);
        // Clearing map
        students.clear();
        System.out.println("After clear(): " + students);
    }
}
```

Compilation & Execution Commands :

javac HashMapDemo.java

java HashMapDemo

Output screenshot:



Result : Thus, the important methods of the HashMap class were successfully demonstrated.

11. LinkedHashMap

Title : Demonstration of LinkedHashMap Methods in Java

Aim : To write a Java program to demonstrate the important methods of the LinkedHashMap class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

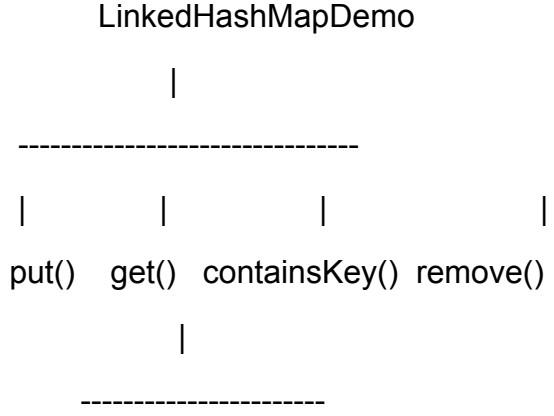
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a LinkedHashMap.
2. Add entries using put().
3. Retrieve a value using get().
4. Check a key using containsKey().
5. Display keys using keySet().
6. Display values using values().
7. Display mappings using entrySet().
8. Remove an entry using remove().

UML Diagram :



```
          |                |             |
       keySet()   values()   entrySet()
```

Source Code :

```
import java.util.LinkedHashMap;
public class LinkedHashMapDemo {
    public static void main(String[] args) {
        LinkedHashMap<Integer, String> employees = new LinkedHashMap<>();
        // Adding key-value pairs
        employees.put(101, "Arjun");
        employees.put(102, "Divya");
        employees.put(103, "Nikhil");
        System.out.println("LinkedHashMap: " + employees);
        // Getting a value
        System.out.println("Value for key 102: "
                + employees.get(102));
        // Checking a key
        System.out.println("Contains key 101: " + employees.containsKey(101));
      // Displaying keys
        System.out.println("Keys: " + employees.keySet());
        // Displaying values
        System.out.println("Values: " + employees.values());
        // Displaying entries
        System.out.println("Entries: " + employees.entrySet());
        // Removing an entry
        employees.remove(103);
        System.out.println("After remove(): " + employees);
    }
```

}

Compilation & Execution Commands :

javac LinkedHashMapDemo.java

java LinkedHashMapDemo

Output screenshot:



Result :

Thus, the important methods of the LinkedHashMap class were successfully demonstrated.

12. TreeMap :

Title : Demonstration of TreeMap Methods in Java

Aim : To write a Java program to demonstrate the important methods of the TreeMap class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware

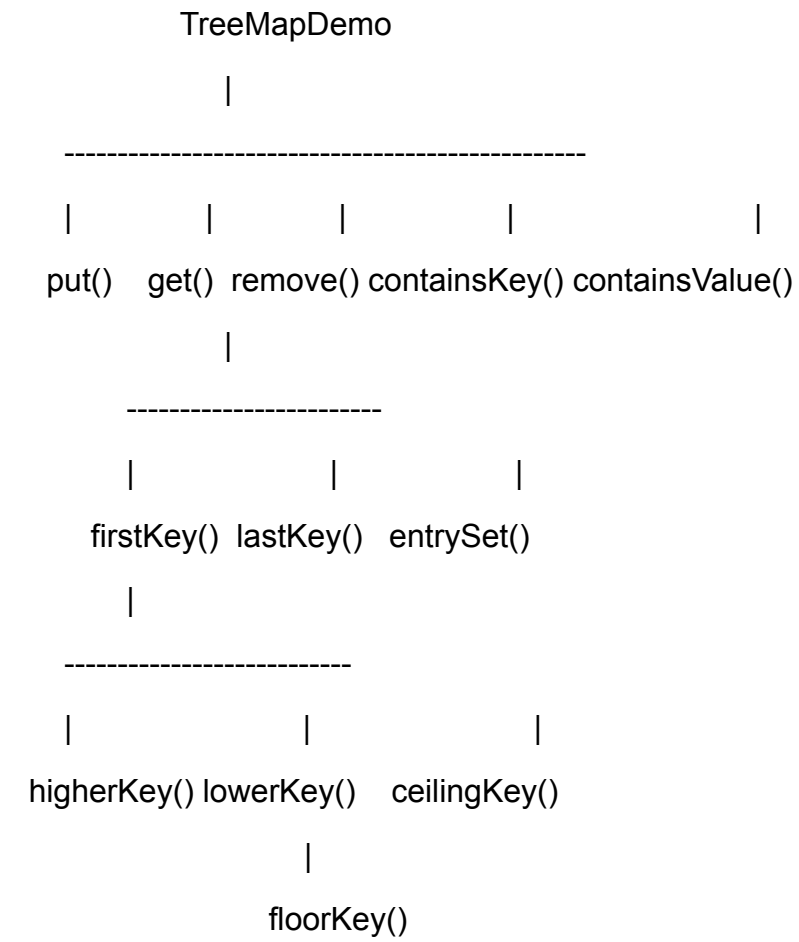
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a TreeMap.
2. Add key-value pairs using put().
3. Retrieve a value using get().
4. Check keys and values using containsKey() and containsValue().
5. Find the first and last keys.
6. Find higher and lower keys.
7. Find ceiling and floor keys.
8. Display entries using entrySet().
9. Remove a key.

UML Diagram:



Source Code :

```
import java.util.TreeMap;
public class TreeMapDemo {
    public static void main(String[] args) {
```

```
TreeMap<Integer, String> students = new TreeMap<>();
  // Adding key-value pairs
  students.put(103, "Meera");
  students.put(101, "Ananya");
  students.put(105, "Rahul");
  students.put(102, "Kiran");
  students.put(104, "Divya");
  System.out.println("TreeMap: " + students);
  // Getting a value
  System.out.println("Value for key 102: " +students.get(102));
  // Checking key and value
  System.out.println("Contains key 103: "
        + students.containsKey(103));
  System.out.println("Contains value Rahul: "
        + students.containsValue("Rahul"));
  // First and last keys
  System.out.println("First Key: " + students.firstKey());
  System.out.println("Last Key: " + students.lastKey());
  // Higher and lower keys
  System.out.println("Higher Key than 102: "
        + students.higherKey(102));
  System.out.println("Lower Key than 102: "
        + students.lowerKey(102));
  // Ceiling and floor keys
  System.out.println("Ceiling Key of 103: "
        + students.ceilingKey(103));
  System.out.println("Floor Key of 103: "
```

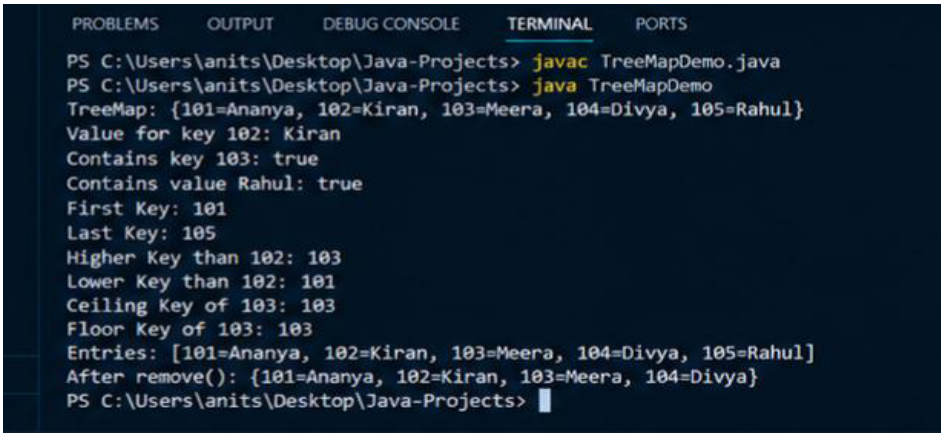
```
            + students.floorKey(103));
        // Displaying entries
        System.out.println("Entries: " + students.entrySet());
        // Removing an entry
        students.remove(105);
        System.out.println("After remove(): " + students);
    }
}
```

Compilation & Execution Commands :

javac TreeMapDemo.java

java TreeMapDemo

Output screenshot:



Result : Thus, the important methods of the TreeMap class were successfully demonstrated.

13. Hashtable

Title : Demonstration of Hashtable Methods in Java

Aim : To write a Java program to demonstrate the important methods of the Hashtable class.

Software requirements:

Software:

Java JDK

Eclipse IDE / Command Prompt

Hardware :

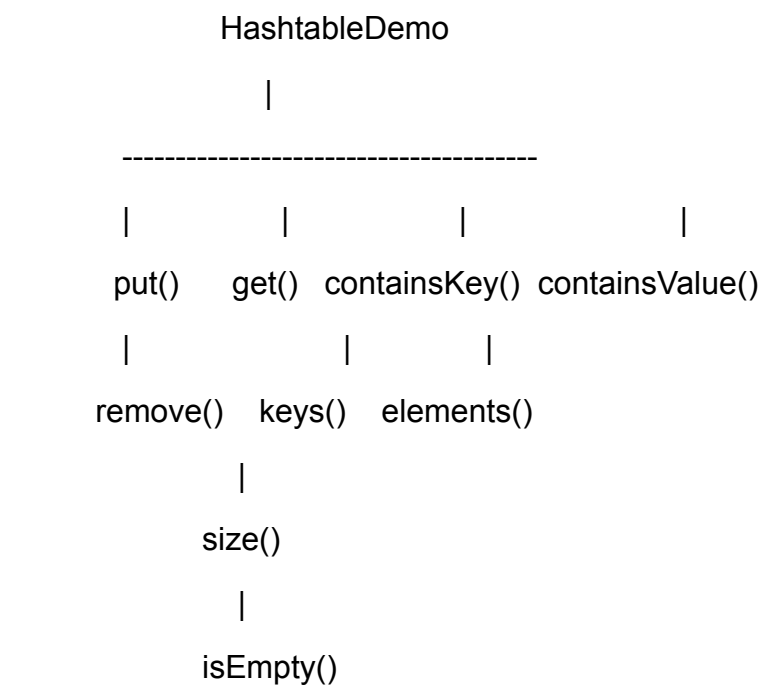
Computer/Laptop

Minimum 4 GB RAM

Algorithm :

1. Create a Hashtable.
2. Add key-value pairs using put().
3. Retrieve a value using get().
4. Check keys and values using containsKey() and containsValue().
5. Display keys using keys().
6. Display values using elements().
7. Find the size using size().
8. Check whether the table is empty using isEmpty().
9. Remove an entry using remove().

UML Diagram :



Source Code :

```
import java.util.Enumeration;
```

```
import java.util.Hashtable;
public class HashtableDemo {
    public static void main(String[] args) {
        Hashtable<Integer, String> students = new Hashtable<>();
        // Adding key-value pairs
        students.put(101, "Aarav");
        students.put(102, "Sneha");
        students.put(103, "Vikram");
        System.out.println("Hashtable: " + students);
        // Getting a value
        System.out.println("Value for key 102: “    + students.get(102));
       // Checking key and value
        System.out.println("Contains key 101: "   + students.containsKey(101));
       System.out.println("Contains value Sneha: "    + students.containsValue("Sneha"));
        // Displaying keys using Enumeration
        System.out.print("Keys: ");
        Enumeration<Integer> keys = students.keys();
        while (keys.hasMoreElements()) {
            System.out.print(keys.nextElement() + " ");
        }
        System.out.println();
        // Displaying values using Enumeration
        System.out.print("Values: ");
        Enumeration<String> values = students.elements();
        while (values.hasMoreElements()) {
            System.out.print(values.nextElement() + " ");
        }
```

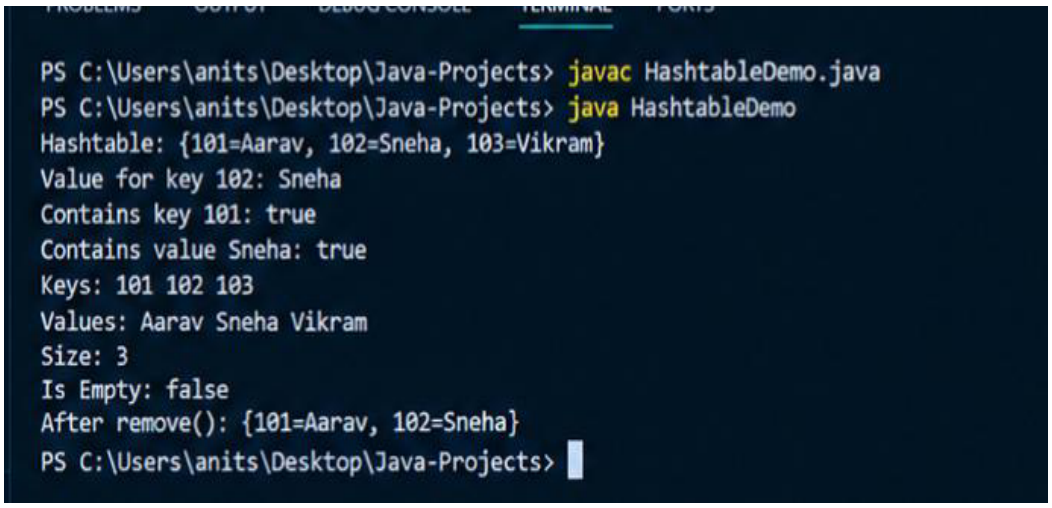
```
        System.out.println();
        // Size and empty check
        System.out.println("Size: " + students.size());
        System.out.println("Is Empty: " + students.isEmpty());
        // Removing an entry
        students.remove(103);
        System.out.println("After remove(): " + students);
    }
}
```

Compilation & Execution Commands :

javac HashtableDemo.java

java HashtableDemo

Output screenshot:

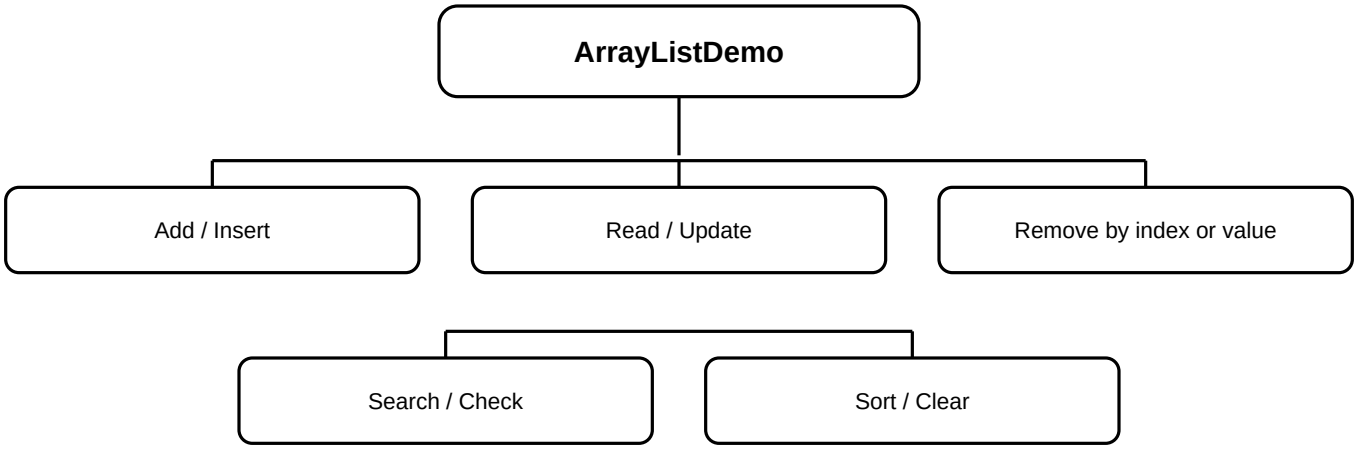


Result : Thus, the important methods of the Hashtable class were successfully demonstrated.

Task 6 — Revised UML Diagrams & Outputs

1. ArrayList

UML Diagram — revised layout



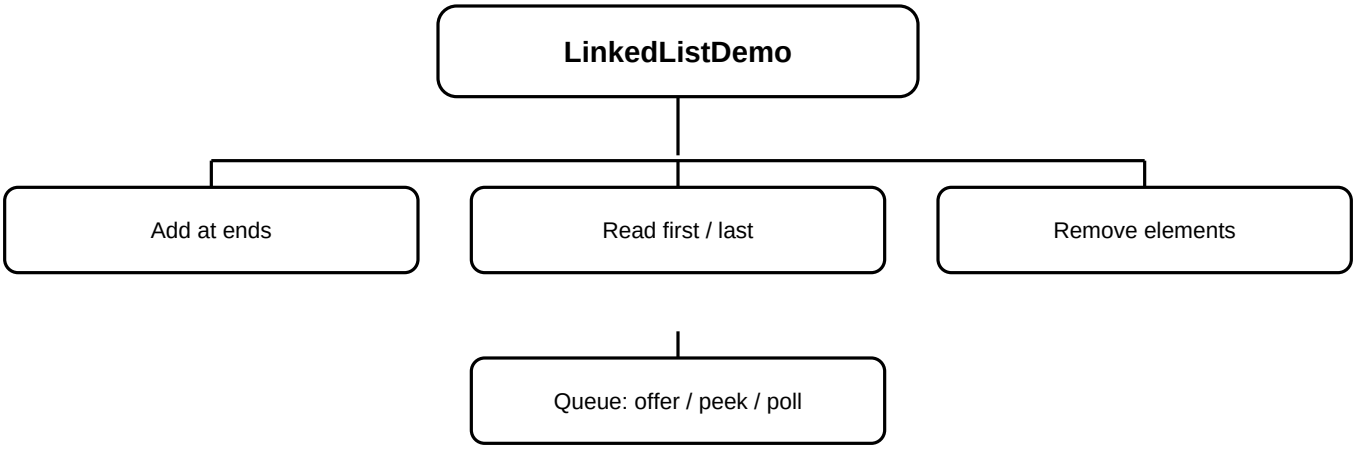
Console Output (sample)

```
Original List: [30, 10, 20, 10]
After add(index, element): [30, 40, 10, 20, 10]
Element at index 2: 10
After set(): [30, 40, 50, 20, 10]
After remove(index): [30, 50, 20, 10]
After remove(object): [30, 50, 20]
Contains 50: true
Size: 3
Is Empty: false
First index of 10: -1
Last index of 10: -1
Sorted List: [20, 30, 50]
After clear(): []
```

Task 6 — Revised UML Diagrams & Outputs

2. LinkedList

UML Diagram — revised layout



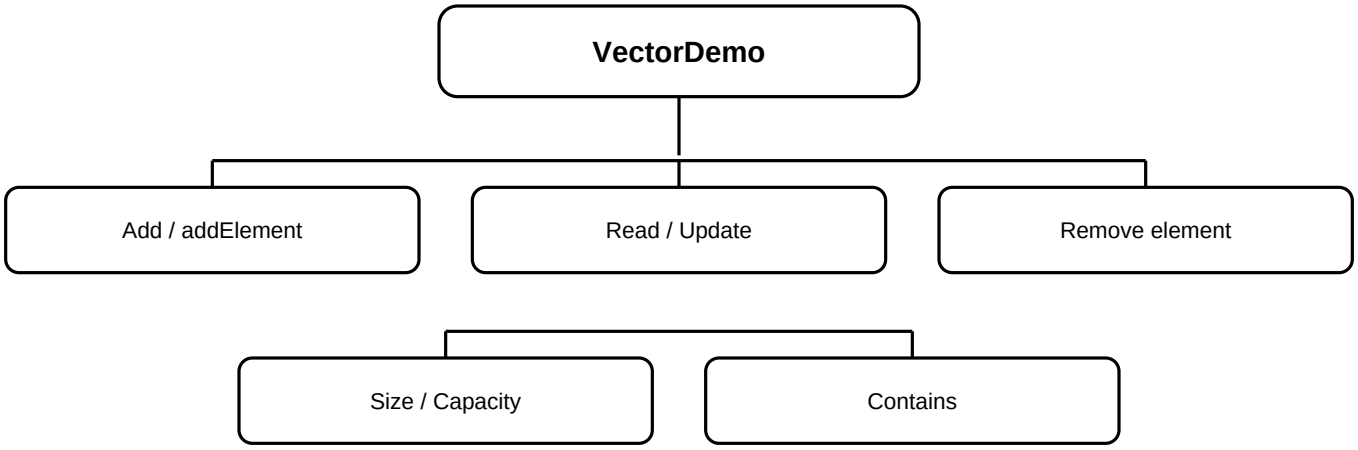
Console Output (sample)

```
Original List: [Anu, Kiran, Meena]
After addFirst() and addLast(): [Rahul, Anu, Kiran, Meena, Priya]
Element at index 2: Kiran
First Element: Rahul
Last Element: Priya
After remove(index): [Rahul, Anu, Meena, Priya]
After remove(object): [Rahul, Anu, Meena, Priya]
After removeFirst(): [Anu, Meena, Priya]
After removeLast(): [Anu, Meena]
After offer(): [Anu, Meena, Sita, Ravi]
Peek: Anu
Poll: Anu
After poll(): [Meena, Sita, Ravi]
```

Task 6 — Revised UML Diagrams & Outputs

3. Vector

UML Diagram — revised layout



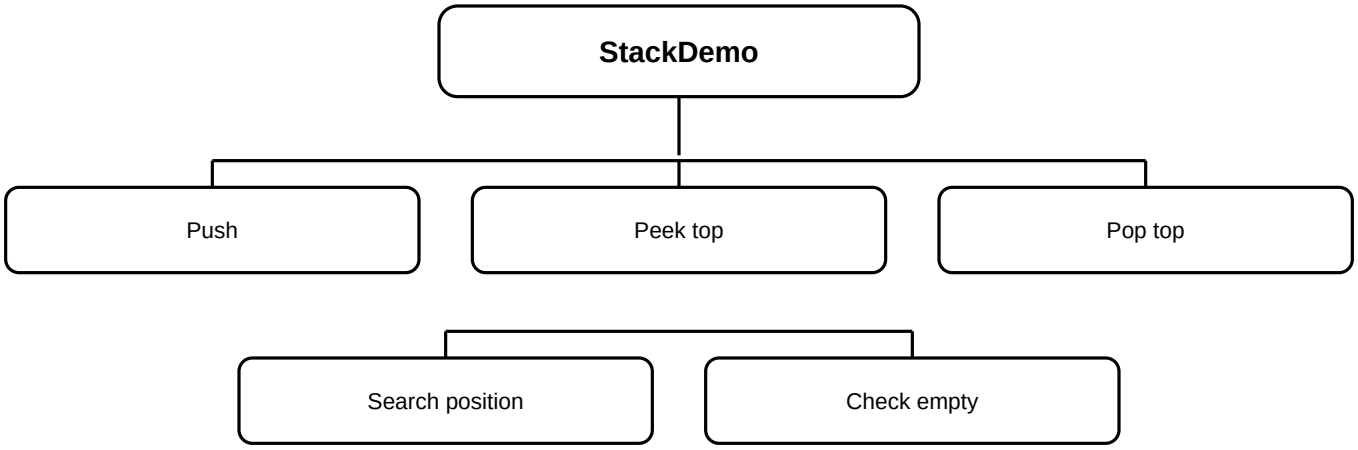
Console Output (sample)

```
Vector: [Java, C, Python]
Element at index 1: C
After set(): [Java, C++, Python]
After remove(index): [C++, Python]
After removeElement(): [C++, JavaScript]
Size: 2
Capacity: 10
Contains JavaScript: true
```

Task 6 — Revised UML Diagrams & Outputs

4. Stack

UML Diagram — revised layout



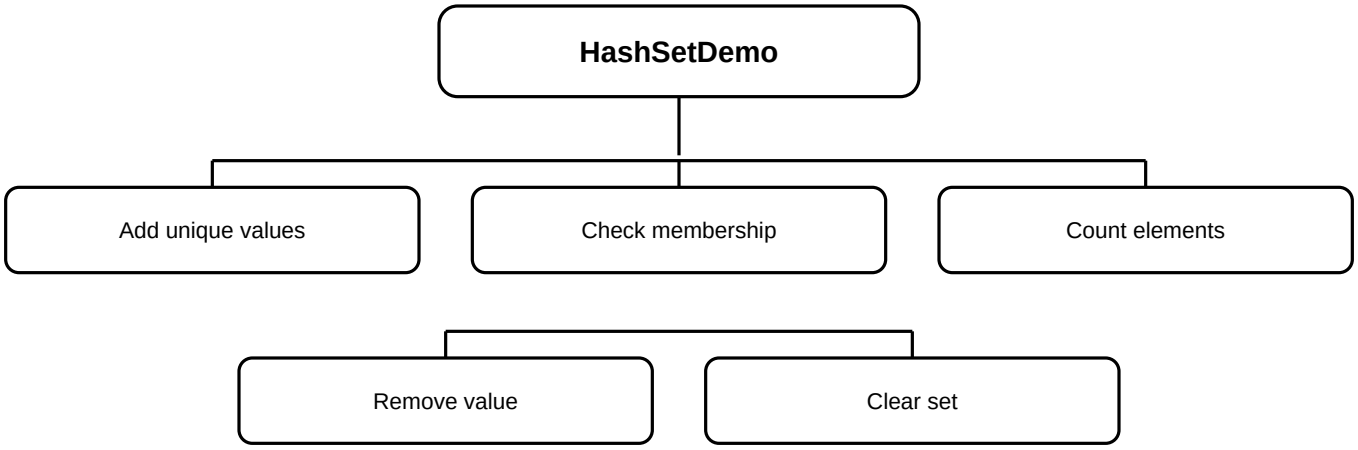
Console Output (sample)

```
Stack: [10, 20, 30]
Top element: 30
Popped element: 30
Stack after pop: [10, 20]
Is Empty: false
Position of 10: 2
```

Task 6 — Revised UML Diagrams & Outputs

5. HashSet

UML Diagram — revised layout



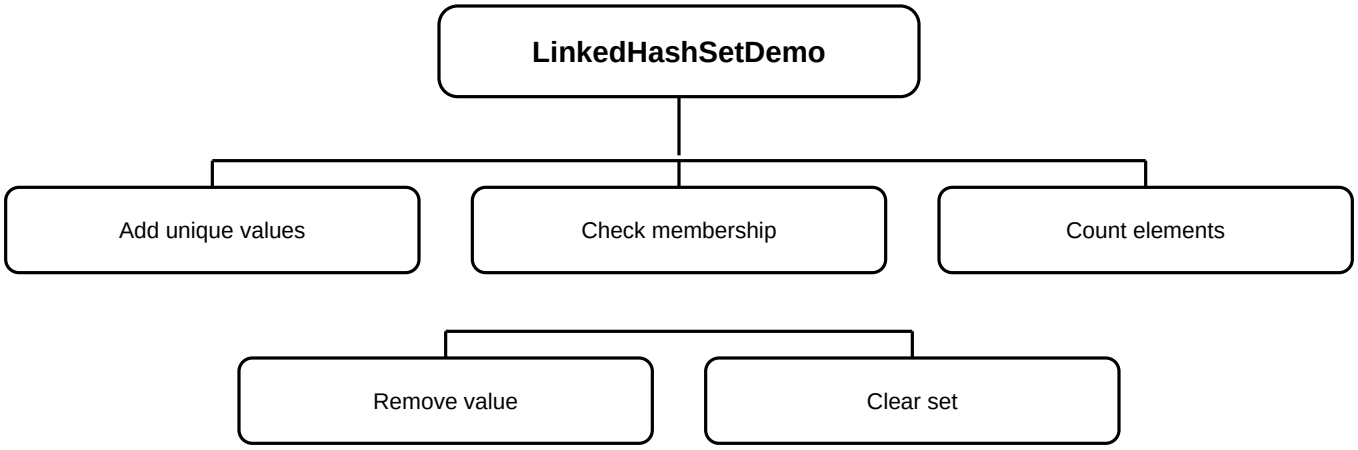
Console Output (sample)

```
HashSet: [Java, Python, C++]
Contains Python: true
Size: 3
Is Empty: false
After remove(): [Java, C++]
After clear(): []
```

Task 6 — Revised UML Diagrams & Outputs

6. LinkedHashSet

UML Diagram — revised layout



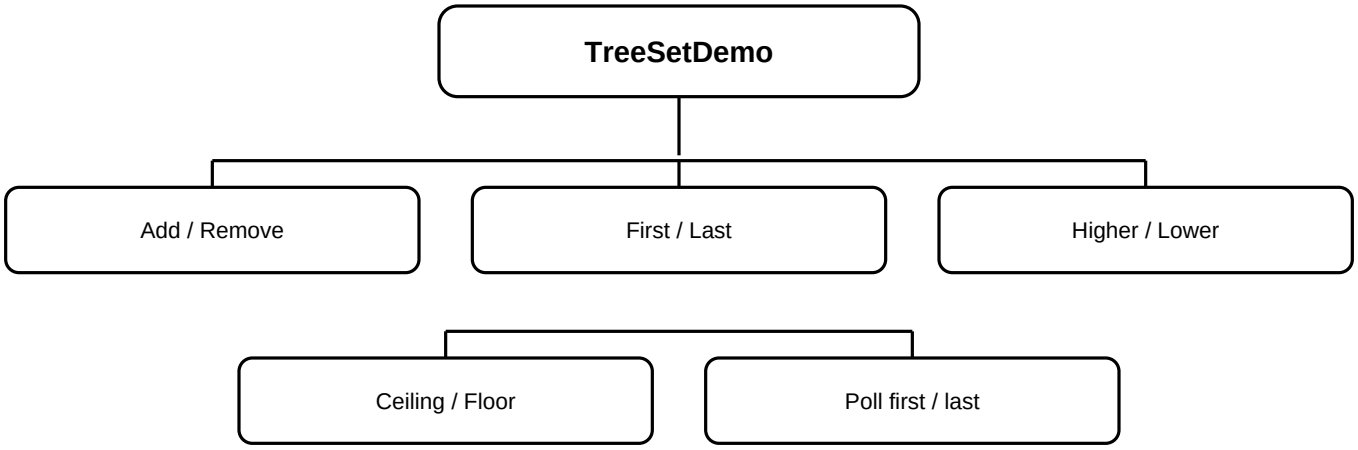
Console Output (sample)

```
LinkedHashSet: [Apple, Banana, Mango]
Contains Banana: true
Size: 3
After remove(): [Apple, Mango]
After clear(): []
```

Task 6 — Revised UML Diagrams & Outputs

7. TreeSet

UML Diagram — revised layout



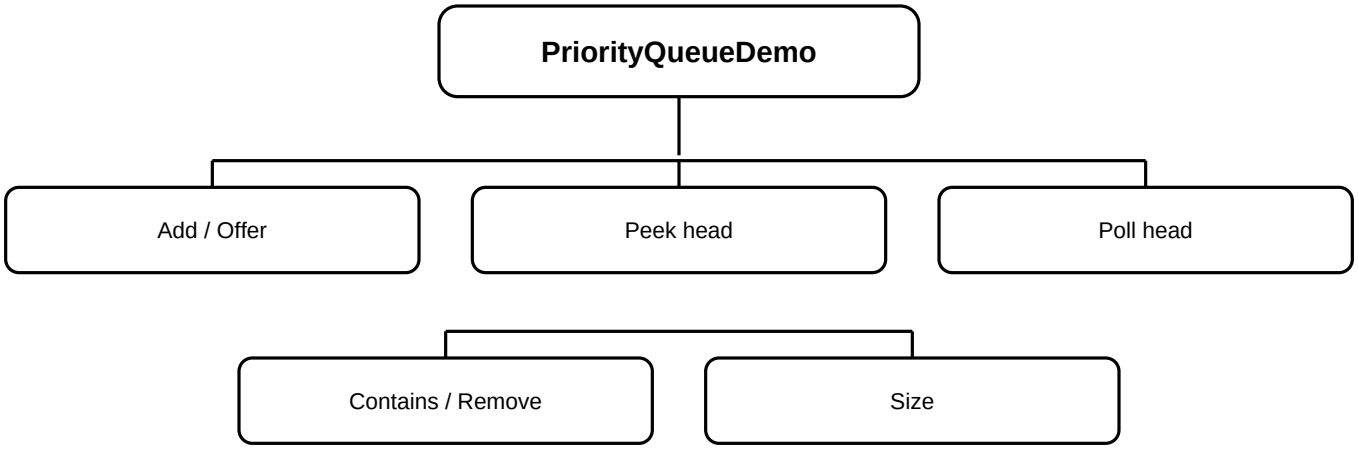
Console Output (sample)

```
TreeSet: [10, 20, 25, 30, 40]
Contains 25: true
After remove(20): [10, 25, 30, 40]
First element: 10
Last element: 40
Higher than 30: 40
Lower than 30: 25
Ceiling of 25: 25
Floor of 25: 25
After pollFirst(): [25, 30, 40]
After pollLast(): [25, 30]
```

Task 6 — Revised UML Diagrams & Outputs

8. PriorityQueue

UML Diagram — revised layout



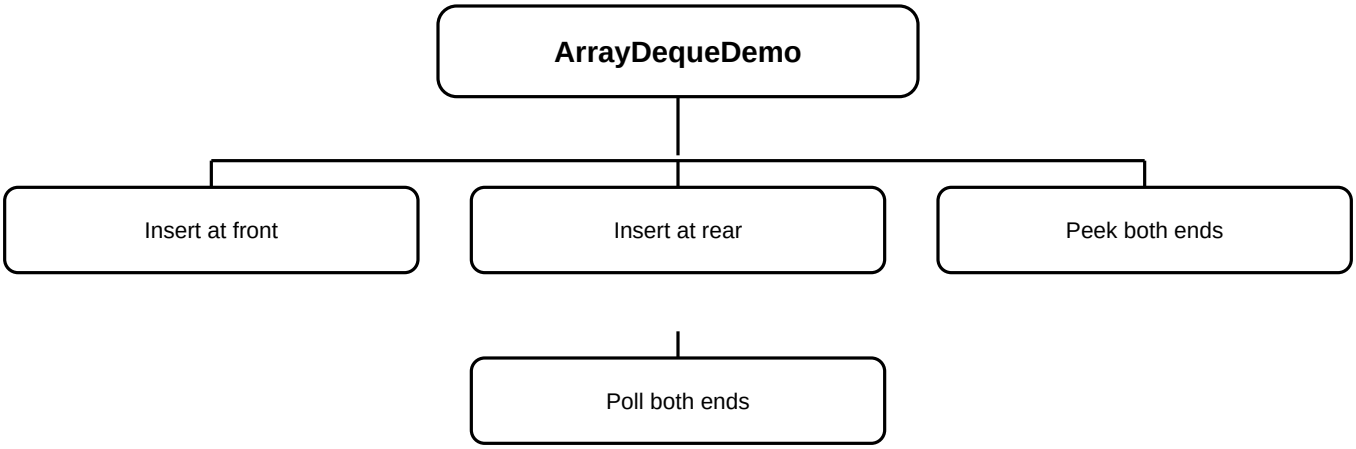
Console Output (sample)

```
PriorityQueue: [10, 20, 30, 40]
Head element: 10
Removed head: 10
Contains 30: true
After remove(30): [20, 40]
Size: 2
```

Task 6 — Revised UML Diagrams & Outputs

9. ArrayDeque

UML Diagram — revised layout



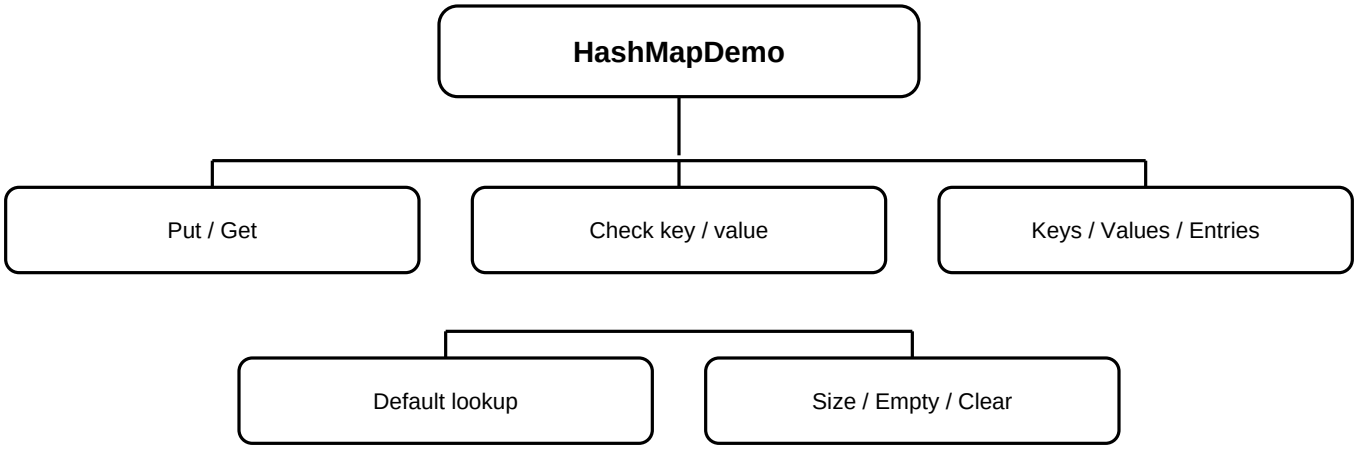
Console Output (sample)

```
Initial deque: [Java, Python]
After addFirst/addLast: [C, Java, Python, JavaScript]
After offer methods: [C++, C, Java, Python, JavaScript, SQL]
First element: C++
Last element: SQL
Removed first: C++
Removed last: SQL
Final deque: [C, Java, Python, JavaScript]
```

Task 6 — Revised UML Diagrams & Outputs

10. HashMap

UML Diagram — revised layout



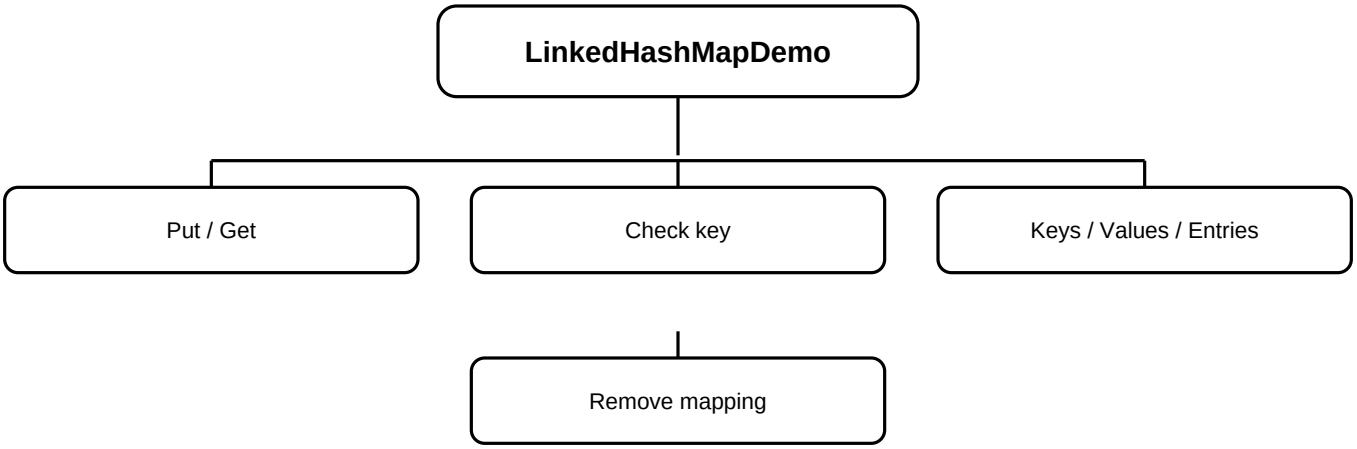
Console Output (sample)

```
HashMap: {101=Ananya, 102=Kiran, 103=Meera}
Value for key 102: Kiran
Contains key 101: true
Contains value Meera: true
Keys: [101, 102, 103]
Values: [Ananya, Kiran, Meera]
Entries: [101=Ananya, 102=Kiran, 103=Meera]
Key 105: Not Found
Size: 3
Is Empty: false
After remove(): {101=Ananya, 102=Kiran}
After clear(): {}
```

Task 6 — Revised UML Diagrams & Outputs

11. LinkedHashMap

UML Diagram — revised layout



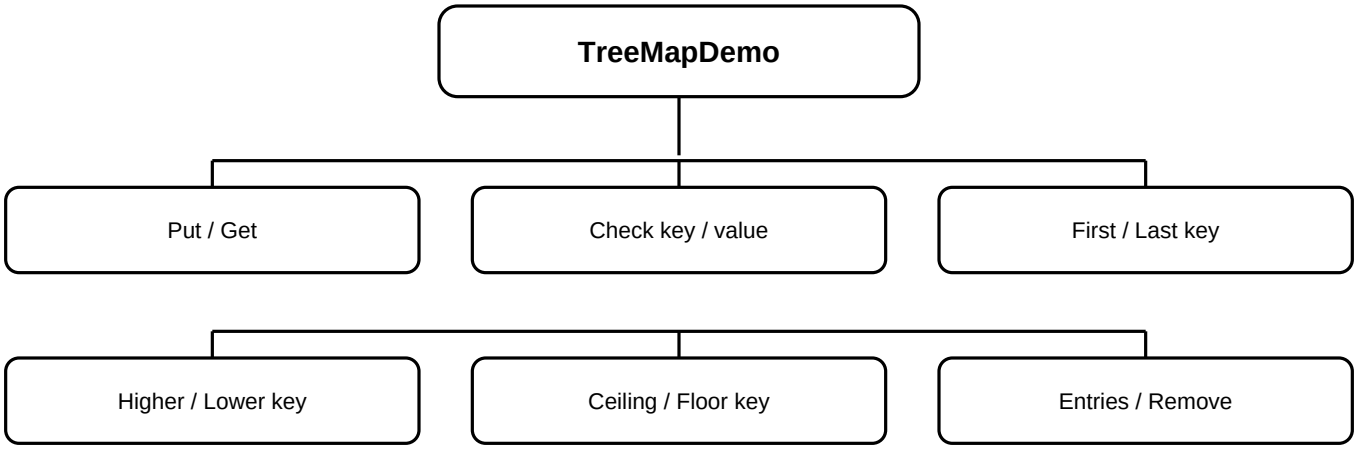
Console Output (sample)

```
LinkedHashMap: {101=Arjun, 102=Divya, 103=Nikhil}
Value for key 102: Divya
Contains key 101: true
Keys: [101, 102, 103]
Values: [Arjun, Divya, Nikhil]
Entries: [101=Arjun, 102=Divya, 103=Nikhil]
After remove(): {101=Arjun, 102=Divya}
```

Task 6 — Revised UML Diagrams & Outputs

12. TreeMap

UML Diagram — revised layout



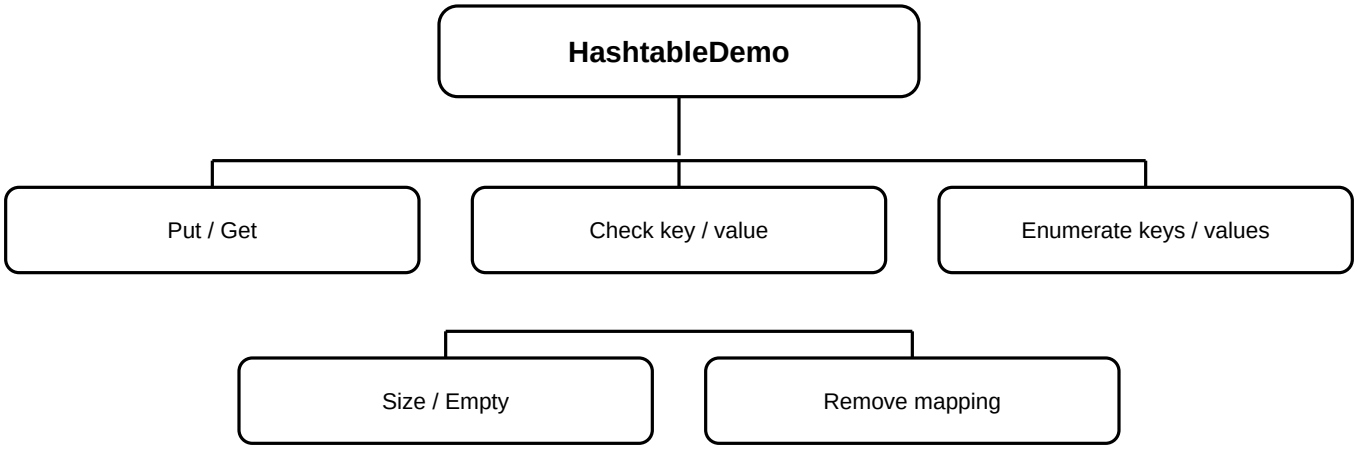
Console Output (sample)

```
TreeMap: {101=Ananya, 102=Kiran, 103=Meera, 104=Divya, 105=Rahul}
Value for key 102: Kiran
Contains key 103: true
Contains value Rahul: true
First Key: 101
Last Key: 105
Higher Key than 102: 103
Lower Key than 102: 101
Ceiling Key of 103: 103
Floor Key of 103: 103
Entries: [101=Ananya, 102=Kiran, 103=Meera, 104=Divya, 105=Rahul]
After remove(): {101=Ananya, 102=Kiran, 103=Meera, 104=Divya}
```

Task 6 — Revised UML Diagrams & Outputs

13. Hashtable

UML Diagram — revised layout



Console Output (sample)

```
Hashtable: {103=Vikram, 102=Sneha, 101=Aarav}
Value for key 102: Sneha
Contains key 101: true
Contains value Sneha: true
Keys: 103 102 101
Values: Vikram Sneha Aarav
Size: 3
Is Empty: false
After remove(): {102=Sneha, 101=Aarav}
```